

Reconocimiento Robusto de la Lengua de Señas Argentina (LSA): de la clasificación estática CNN a la detección espacial con YOLO

Julián Santiago González Avendaño
Agustín García Smith

Dirección: Ing. Roxana Martínez

Mayo 2026

Resumen

El reconocimiento automático de configuraciones manuales (*handshapes*) es el cuello de botella de los sistemas de reconocimiento de la Lengua de Señas. El trabajo de Quiroga et al. (2017) reporta **96.18 %** de *accuracy* usando una red neuronal convolucional (CNN) tipo LeNet sobre el dataset LSA16 con preprocesamiento manual de segmentación y alineamiento canónico, pero ese pipeline depende fuertemente de recortar la mano antes de clasificar. En este trabajo (i) replicamos el baseline del paper identificando el preprocesamiento canónico como ingrediente crítico no documentado explícitamente, (ii) mejoramos el baseline a **97.62 % $\pm$ 1.63 %** mediante *transfer learning* con ResNet-18 (un modelo más moderno preentrenado en millones de imágenes), y (iii) proponemos como contribución original un detector espacial basado en YOLOv8 que opera sobre la imagen completa sin preprocesamiento y alcanza **89.74 %** de *accuracy* y **mAP@50 = 0.923**. La comparación crítica muestra que sobre la imagen sin preprocesar (escenario realista de webcam), el CNN clásico colapsa al 22.12 % mientras YOLO se mantiene en $\sim 90 %$, una diferencia de **+67 puntos porcentuales** que valida el cambio de paradigma de clasificación a detección espacial para sistemas reales.

Palabras clave: Lengua de Señas Argentina, reconocimiento de *handshapes*, redes neuronales convolucionales, detección de objetos, YOLO, transfer learning, LSA16.

Nota al lector. Este documento está pensado para una audiencia que no necesariamente tiene formación previa en *machine learning* o redes neuronales. La Sección 2 (“Conceptos previos”) introduce desde cero los términos técnicos centrales del trabajo. Los lectores familiarizados pueden saltarla sin perder el hilo.

1. Introducción

1.1. Contexto: Lengua de Señas y reconocimiento automático

La Lengua de Señas Argentina (LSA) es la lengua natural de la comunidad sorda en Argentina. A diferencia del español hablado, se articula con configuraciones de las manos (*handshapes*),

su movimiento, su posición relativa al cuerpo, y expresiones faciales. Construir un sistema que “traduzca” video de LSA a texto en español permitiría reducir barreras de comunicación entre personas sordas y oyentes, especialmente en contextos como atención médica, educación o trámites administrativos.

El problema completo de *Sign Language Recognition* (SLR) involucra varias etapas: localizar las manos en cada frame, clasificar su forma estática (*handshape*), seguir su trayectoria en el tiempo, interpretar expresiones faciales, y finalmente asignar significado semántico a la secuencia [5]. Investigaciones previas mostraron que el reconocimiento correcto de la *forma de la mano* es el factor más crítico para que todo el pipeline funcione [4].

Este trabajo se enfoca específicamente en la primera tarea: dado una imagen de una persona haciendo una seña estática, identificar cuál de 16 *handshapes* prototípicos está usando. Trabajamos con el dataset público **LSA16** [2], formado por 800 imágenes de 10 sujetos haciendo 16 configuraciones manuales diferentes, 5 veces cada uno.

1.2. El problema central

Los trabajos publicados en LSA16 (incluido el *benchmark* de referencia de Quiroga et al. [1], que reporta 96.18% de *accuracy*) asumen, casi sin excepción, que la imagen de entrada al clasificador es un *recorte limpio y centrado* de la mano sobre fondo negro. Es decir: alguien (o un algoritmo de visión clásica) ya recortó la mano del resto de la imagen antes de que el modelo neural empiece a clasificar.

Pero en una aplicación real –por ejemplo, una persona usando una cámara web de su computadora– la imagen que llega al sistema es la imagen *completa*: el rostro, el torso, ambas manos en algún punto del cuadro, fondo no controlado, iluminación variable. Si el sistema espera un crop limpio, alguien tiene que producir ese crop antes de invocarlo, y ese paso suele ser frágil.

En este trabajo argumentamos y validamos que un cambio de paradigma resuelve esto: usar un modelo de **detección de objetos** (en particular YOLO [11]) que recibe la imagen completa y produce simultáneamente (a) dónde está la mano y (b) qué configuración hace, todo en un solo modelo aprendido.

1.3. Pregunta de investigación

¿En qué medida mejora la aplicabilidad real del reconocimiento de LSA al transicionar de un modelo de clasificación estática (CNN sobre crop preprocesado) a un modelo de detección espacial (YOLO sobre imagen completa)?

1.4. Contribuciones del trabajo

1. Una **replicación reproducible** del baseline del paper de referencia, identificando un paso de preprocesamiento crítico que el paper no documenta explícitamente.
2. Una **caracterización honesta de la varianza** del experimento, mostrando que el promediado de muchas corridas oculta inestabilidad sustantiva del entrenamiento.
3. Una **mejora cuantitativa** del baseline usando *transfer learning* con ResNet-18, alcanzando $97.62\% \pm 1.63\%$ y superando al paper.
4. Un **pipeline automático** para generar anotaciones de *bounding boxes* (cajas delimitadoras) desde las máscaras de segmentación del dataset.
5. Una **evaluación rigurosa** de YOLOv8 sobre el escenario realista (imagen completa sin preprocesar), alcanzando 89.74% de *accuracy* y $\text{mAP}@50 = 0.923$.

6. Una **ablación negativa** con un modelo de mayor capacidad, que aporta un resultado científicamente útil: en datasets pequeños, escalar el modelo no siempre mejora la clasificación.

2. Conceptos previos

Esta sección introduce los conceptos técnicos centrales del trabajo. Si el lector ya conoce *deep learning*, puede saltarla.

2.1. Aprendizaje supervisado y redes neuronales

Una **red neuronal** es una función matemática parametrizada por muchos números (los *pesos*), que aprende a hacer una tarea ajustando esos pesos a partir de ejemplos. En aprendizaje supervisado, la tarea se aprende minimizando una **función de pérdida** (*loss*) que mide la diferencia entre las predicciones de la red y las respuestas correctas (*ground truth*) en un dataset de entrenamiento.

Ejemplo simple. Si queremos una red que distinga gatos de perros, entrenamos con miles de imágenes etiquetadas como “gato” o “perro”. La red empieza con pesos aleatorios y va ajustándolos para que sus predicciones se acerquen cada vez más a las etiquetas correctas. Este ajuste se hace con el algoritmo *gradient descent*: en cada paso, los pesos se mueven en la dirección que más reduce la pérdida.

Los hiperparámetros de entrenamiento incluyen:

- **Learning rate:** cuán grandes son los pasos de ajuste. Demasiado grande $\rightarrow$ la red no converge. Demasiado chico $\rightarrow$ tarda mucho.
- **Batch size:** cuántos ejemplos procesa antes de ajustar los pesos. Típicamente 16, 32 o 64.
- **Épocas:** cuántas veces la red ve cada ejemplo del dataset.

2.2. Redes neuronales convolucionales (CNN)

Una **CNN** es un tipo especial de red neuronal diseñada para imágenes. Su componente clave es la **operación de convolución**: pequeños filtros (típicamente 3×3 píxeles) “escanean” la imagen detectando patrones locales como bordes, esquinas o texturas.

Intuición clave. Las primeras capas de una CNN detectan patrones simples (bordes verticales, contrastes, manchas de color). Las capas siguientes combinan estos patrones para detectar elementos más complejos (curvas, formas, partes de la mano). Las capas finales combinan todo para reconocer el objeto entero (la configuración manual completa).

Componentes típicos de una CNN:

- **Capa convolucional (Conv):** aplica filtros sobre la imagen, produciendo “mapas de características”. Cada filtro aprende a detectar un patrón distinto.
- **Función de activación** (típicamente ReLU o ELU): introduce no-linealidad, permitiendo que la red aprenda relaciones complejas.
- **Max-pooling:** reduce el tamaño espacial tomando el valor máximo en regiones de 2×2 . Hace al modelo más invariante a pequeños desplazamientos.

- **Capa fully-connected (Linear)**: al final, una capa densa combina toda la información para producir las probabilidades de cada clase.

La arquitectura **LeNet** [6] (1998) fue una de las primeras CNNs exitosas. **VGG** [7], **ResNet** [9] (*Residual Networks*) e **Inception** [10] son arquitecturas más modernas que mejoran resultados pero requieren más cómputo. En este trabajo usamos LeNet (réplica del paper de referencia) y ResNet-18 (nuestra mejora).

2.3. Batch normalization

Durante el entrenamiento de una red profunda, las distribuciones de activaciones (valores en cada capa) cambian a medida que se actualizan los pesos, fenómeno conocido como *internal covariate shift*. Esto desestabiliza el entrenamiento y requiere learning rates más pequeños.

La **Batch Normalization** [18] resuelve esto normalizando las activaciones de cada capa para que en cada *batch* tengan media cero y desvío estándar uno. Esto acelera y estabiliza el entrenamiento. Lo usamos en todas nuestras CNNs.

2.4. Transfer learning

El **transfer learning** es una técnica que aprovecha modelos *ya entrenados* en una tarea grande (típicamente clasificación de millones de imágenes generales en ImageNet [16]) y los reusa para tareas específicas con menos datos.

£**Por qué funciona?** Las primeras capas de una CNN aprenden cosas *universales* sobre imágenes (cómo detectar bordes, texturas, formas elementales). Estas características son útiles tanto para distinguir gatos de perros como para clasificar configuraciones manuales. Reusar esas capas en lugar de entrenarlas desde cero es como *empezar la tarea con conocimiento previo*, no desde la ignorancia total.

En la práctica, transfer learning consiste en:

1. Cargar un modelo (por ejemplo ResNet-18) con sus pesos preentrenados en ImageNet.
2. Reemplazar la última capa (que predecía las 1000 clases de ImageNet) por una nueva capa para nuestras 16 clases de LSA16.
3. Entrenar (*fine-tune*) el modelo en nuestro dataset, ajustando finamente los pesos preentrenados con un *learning rate* pequeño para no romper lo que ya sabe.

2.5. Clasificación vs. detección de objetos

Es fundamental entender la diferencia entre estas dos tareas, porque el trabajo se basa en transicionar de una a la otra.

Clasificación de imágenes. Dada una imagen, la salida es una *etiqueta* (por ejemplo, “configuración manual: V”). El modelo asume que la imagen contiene el objeto centrado y bien recortado. **No localiza nada**; solo dice qué hay. Es la tarea que resuelve LeNet sobre los crops del LSA16.

Detección de objetos. Dada una imagen, la salida es una lista de *objetos detectados*, cada uno con (a) una **caja delimitadora** (*bounding box*) que indica dónde está, y (b) la clase a la que pertenece. La detección se hace sobre la imagen *completa* sin preprocesar. Es la tarea que resuelve YOLO.

Por qué importa para este trabajo. En el escenario real (webcam), el sistema no tiene un crop limpio: tiene la imagen completa con la persona, ambas manos, el fondo. Un clasificador puro no puede manejar esto porque le falta el paso de “encontrar dónde mirar”. Un detector como YOLO sí puede.

2.6. YOLO: detector de objetos en una sola etapa

YOLO (*You Only Look Once* [11]) es una familia de modelos de detección de objetos. La idea central es que la imagen completa pasa una sola vez por una red convolucional que produce *directamente*: las coordenadas ($x_c, y_c, ancho, alto$) de cada bounding box, y la clase y confianza asociadas. Comparado con métodos previos de detección que hacían varios pasos secuenciales (proposición de regiones, luego clasificación), YOLO es mucho más rápido manteniendo buena precisión.

En este trabajo usamos **YOLOv8** [14] (versión 2023 de Ultralytics), que es el estado del arte abierto al momento del trabajo. Viene en cinco tamaños desde **nano** (3M parámetros, muy rápido) hasta **x-large** (68M parámetros, más preciso pero lento).

2.7. Métricas de evaluación

2.7.1. Accuracy

Métrica clásica de clasificación: porcentaje de predicciones correctas sobre el total. Si tenemos 100 imágenes y el modelo clasifica 95 correctamente, $accuracy = 95\%$.

2.7.2. IoU (Intersection over Union)

Métrica de localización: dada una *bounding box* predicha por el modelo y la *bounding box* real (*ground truth*), IoU mide **cuánto se solapan**:

$$IoU = \frac{\text{Área de intersección}}{\text{Área de unión}}$$

$IoU = 1$ significa que las dos cajas son idénticas. $IoU = 0$ significa que no se solapan en absoluto. En la práctica, se considera que una detección es “correcta” si $IoU > 0.5$ con la caja real.

2.7.3. Precision y Recall

Precision = de todas las detecciones que hizo el modelo, ¿qué fracción es correcta?

Recall = de todos los objetos que existían (*ground truth*), ¿qué fracción detectó el modelo?

Idealmente queremos ambos altos, pero suele haber un trade-off: ser muy estricto sube precision pero baja recall, y viceversa.

2.7.4. mAP (mean Average Precision)

Métrica estándar de detección de objetos, combina precision y recall a distintos umbrales de IoU:

- **mAP@50**: promedio de precision considerando una detección correcta si $IoU \geq 0.5$. Es la métrica más usada.
- **mAP@50-95**: promedio sobre múltiples umbrales (0.5, 0.55, ..., 0.95). Más estricta. Es la métrica principal de los benchmarks modernos como COCO.

En este trabajo reportamos ambas para los modelos YOLO.

2.8. Optimización: paisaje de pérdida y basins de atracción

Al entrenar una red, lo que hace el optimizador (Adam, SGD, etc.) es minimizar la función de pérdida ajustando millones de pesos. Conceptualmente, esto puede pensarse como “rodar una pelota cuesta abajo” en un paisaje de pérdida de altísima dimensión.

Analogía geográfica. El paisaje de pérdida es como una geografía multidimensional. Cada punto del paisaje es una combinación de valores de los pesos; la altura del paisaje en ese punto es el valor de la pérdida. Entrenar es como soltar una pelota: rueda cuesta abajo siguiendo la pendiente hasta caer al fondo de algún valle.

Un **basin de atracción** (cuenca de atracción) es una zona del paisaje donde toda pelota soltada ahí termina en el mismo punto. La inicialización aleatoria de los pesos es como soltar la pelota en un punto al azar: si caemos en un mal basin (un valle superficial lejos del óptimo real), terminamos con un modelo malo. Si caemos en un buen basin, terminamos con un modelo bueno.

El término “basin” viene de la cuenca hidrográfica en geografía: el área donde toda gota de agua termina en el mismo río.

Esta intuición es importante porque explica:

- Por qué entrenamientos con la misma arquitectura pero distinta semilla aleatoria dan resultados diferentes.
- Por qué el *transfer learning* funciona tan bien: empezar con pesos preentrenados es como soltar la pelota *ya dentro de un buen basin*, no en una posición aleatoria.
- Por qué el *data augmentation* ayuda pero no resuelve el problema raíz: cambia ligeramente la forma del paisaje pero no elimina los malos basins.

2.9. Data augmentation

Cuando el dataset de entrenamiento es chico, una técnica común es **augmentation**: generar variaciones sintéticas de las imágenes durante el entrenamiento (rotaciones, traslaciones, cambios de brillo, ruido). El modelo nunca ve exactamente la misma imagen dos veces, lo que actúa como regularización y suele mejorar generalización.

En este trabajo usamos augmentation de manera *cuidadosa*: específicamente **evitamos el flip horizontal** porque convertir una mano derecha en izquierda rompería la etiqueta del dataset (LSA16 solo anota la mano derecha).

3. Estado del arte

3.1. Reconocimiento de handshapes en LSA

El dataset LSA16 fue introducido por Ronchetti et al. [2] junto con el método **ProbSom**, un clasificador basado en la transformada de Radon (una técnica clásica de procesamiento de imágenes que no usa redes neuronales). ProbSom alcanza 92.30 % de *accuracy* y es la primera línea base publicada sobre el dataset.

Posteriormente, Quiroga et al. [1] compararon varias arquitecturas CNN modernas sobre el mismo dataset (Tabla 1). Su mejor resultado fue VGG-16 con 95.92 %, pero notablemente LeNet –la CNN más simple– alcanzó 95.78 %, casi tan bien como VGG con muchísimos menos parámetros. Por eso lo eligen como baseline “simple pero efectivo” del paper.

El paper también explora el efecto del preprocesamiento manteniendo LeNet fijo (Tabla 2). Aquí se ve que la versión segmentada en color (RGB) da el mejor resultado (96.18 %); las versiones reducidas pierden información útil para discriminar dedos.

Cuadro 1: Resultados del paper de referencia [1] sobre LSA16 segmentado.

Arquitectura	Accuracy
Feedforward (baseline simple, sin convoluciones)	86.58 %
LeNet [6]	95.78 %
All Convolutional [8]	94.56 %
VGG-16 [7]	95.92 %
ResNet-34 [9]	93.49 %
Inception-v3 [10]	91.98 %

Cuadro 2: Efecto del preprocesamiento sobre LeNet, reportado por Quiroga et al. [1].

Preprocesamiento	Accuracy
Raw (mano sin segmentar fondo)	83.54 %
Segmented Hand RGB	96.18 %
Grayscale (escala de grises)	87.08 %
Black and White (binarizado)	91.38 %
Contour only (solo contorno)	80.64 %

3.2. Detección de objetos: del clasificador al detector

Históricamente, los métodos de detección de objetos hacían dos pasos: (a) proponer regiones candidatas con un algoritmo clásico, y (b) clasificar cada región con una CNN. Esto era lento. **YOLO** [11] cambió el paradigma al unificar ambos pasos en una única red que produce todas las detecciones de la imagen en una sola pasada (de ahí el nombre: *You Only Look Once*).

Versiónes sucesivas (YOLOv2 [12], YOLOv3 [13], hasta YOLOv8 [14]) han mejorado precisión y velocidad. YOLOv8 (Ultralytics, 2023) viene preentrenada en el dataset **COCO** [15] (más de un millón de imágenes con 80 categorías comunes). Esto la convierte en una excelente base para transfer learning hacia tareas específicas como nuestro LSA16.

Hasta donde sabemos, no existe en la literatura una evaluación sistemática de YOLO sobre LSA16 que contraste rigurosamente con el baseline CNN clásico. Esto es precisamente lo que este trabajo aporta.

4. Dataset LSA16

4.1. Composición

El dataset LSA16 consta de **800 imágenes** de 10 sujetos que ejecutan 5 repeticiones de cada una de las 16 configuraciones manuales más usadas en LSA. Las imágenes fueron capturadas en condiciones controladas (fondo blanco, ropa negra, iluminación uniforme) y los intérpretes usaron **guantes fluorescentes** de colores saturados para facilitar la segmentación. El nombre de cada archivo sigue el patrón `clase_sujeto_repeticion.png`.

Las 16 clases son: Five (mano abierta), Four (cuatro dedos), Horns (cuernos), Curve (mano curva), Fingers (montoncito), Double, Hook (gancho), Index (índice), L, Flat (mano plana), Mitten (manopla), Beak (pico), Thumb (pulgar), Fist (puño), Telephone (shaka) y V.

4.2. Variantes disponibles

El dataset se distribuye en dos versiones que serán claves para entender los experimentos siguientes:

- `lsa16_raw/`: imagen completa de 640×480 píxeles. Muestra a la persona completa con ambos guantes visibles, fondo blanco y ropa negra. **Es el escenario realista**, parecido a lo que vería una cámara web.
- `lsa16_segmented_right_hand/`: recorte de la mano derecha (la dominante en la mayoría de los sujetos) con el fondo seteado a negro. Tamaño variable, típicamente $\sim 150 \times 150$ píxeles. **Es el input que el paper usa**.

Es importante notar que **las 16 clases corresponden a configuraciones de la mano derecha únicamente**. La mano izquierda (con guante de color distinto, típicamente cian) aparece en algunas imágenes en posición no-dominante pero no tiene etiqueta de clase asociada.

5. Pipeline CNN baseline

En esta sección replicamos el baseline del paper, identificamos un paso crítico no documentado, y proponemos una mejora con *transfer learning*.

5.1. Réplica del paper y descubrimiento del preprocesamiento canónico

Al entrenar LeNet sobre `lsa16_segmented_right_hand/` con los hiperparámetros exactos del paper (Adam, *learning rate* 0.0007, 20 épocas, sin augmentation), obtuvimos en 3 corridas iniciales **$83.33\% \pm 2.12\%$, 12.85 puntos porcentuales por debajo** del 96.18% reportado. Este gap era demasiado grande para atribuirlo a ruido estadístico.

La inspección visual del dataset reveló la causa: **las manos en las imágenes segmentadas no están en orientación canónica**. Los sujetos pueden tener la mano vertical, inclinada o horizontal, según el sujeto y la repetición. Esa variabilidad espacial le exige al modelo aprender invariancia a rotación a partir de apenas ~ 45 ejemplos por clase, lo que excede su capacidad.

Revisando el paper original del dataset (Ronchetti et al. 2016 [2], referenciado por el paper de 2017), encontramos un **pipeline de normalización geométrica** que el paper de 2017 omite mencionar explícitamente:

1. Cálculo del componente conectado más grande para obtener la máscara binaria limpia de la mano.
2. Cálculo de los ejes principales mediante **PCA**¹ sobre las coordenadas de los píxeles de la mano. La dirección con mayor variabilidad indica hacia dónde apunta la mano.
3. Rotación de la imagen para alinear el eje principal con la vertical (los dedos apuntando hacia arriba).
4. Corrección de inversión 180°: contando “cruces horizontales” (transiciones entre fondo y mano sobre líneas horizontales), determinamos si los dedos quedaron arriba o abajo. El lado con más cruces son los dedos (porque están separados, generando muchas transiciones). Si quedaron abajo, rotamos 180°.
5. Recorte al *bounding box* de la mano y redimensión a 128×128 manteniendo *aspect ratio* (relleno con negro donde haga falta).

Implementamos este pipeline en `src/preprocessing.py`. Al regenerar el dataset y reentrenar LeNet con los mismos hiperparámetros del paper, la *accuracy* sube a **$95.83\% \pm 1.18\%$** en 3 corridas, indistinguible del 96.18% del paper dentro del ruido estadístico. **El gap de 12.85 pp se debía exclusivamente a este preprocesamiento canónico no documentado en el texto del paper de 2017.**

¹**PCA** (*Principal Component Analysis*, Análisis de Componentes Principales) es una técnica estadística que encuentra las direcciones de mayor variabilidad en un conjunto de puntos. Aplicado a los píxeles de una mano, identifica el eje a lo largo del cual la mano está más extendida.

5.2. Arquitectura LeNet usada

Réplica fiel de la especificación del paper:

- Cuatro bloques convolucionales con 32, 64, 128, 256 canales respectivamente.
- Filtros 3×3 con *padding*.
- Cada bloque: Conv $\rightarrow$ ELU [19] $\rightarrow$ MaxPool(2,2) $\rightarrow$ BatchNorm [18].
- Cabeza clasificadora: Flatten $\rightarrow$ Linear(16384, 512) $\rightarrow$ ELU $\rightarrow$ Linear(512, 16).

Total: aproximadamente 8.8 millones de parámetros entrenables.

5.3. Caracterización honesta de la varianza

El paper de referencia reporta el promedio de 100 corridas. Esto es estadísticamente prudente pero *esconde* información importante. Al ampliar nuestro experimento de 3 a 10 corridas observamos una distribución bimodal: la *accuracy* media baja a **91.25 %** y el desvío estándar sube a **7.83 %**. Dos corridas particulares (semillas 8 y 9) produjeron 70 % y 85 % respectivamente, muy por debajo del resto.

El análisis de la matriz de confusión en la semilla 8 reveló el fenómeno: el modelo cayó en un **mode collapse** – predice “Flat” (clase 9) en 17 de los 24 errores, ignorando la mayor parte de las otras clases. No es un problema de imágenes específicas sino del propio proceso de entrenamiento: con esa inicialización aleatoria particular, el optimizador quedó atrapado en un *basin* de mala calidad (ver Sección 2.8) del paisaje de pérdida.

Este resultado tiene una implicación metodológica: el paper, al promediar 100 corridas, *diluye* estos *outliers*. Reportar el promedio sin la varianza puede ocultar inestabilidad práctica relevante.

5.4. Mejoras propuestas

5.4.1. Data augmentation: hipótesis refutada

Hipotetizamos que data augmentation (rotación $\pm 10^\circ$, traslación $\pm 10\%$, jitter de color leve) actuaría como regularización y reduciría la varianza al estabilizar el entrenamiento. El experimento sobre 10 corridas dio **91.62 % $\pm$ 6.71 %**: la media casi no cambió (+0.37 pp) y la varianza apenas bajó.

Más interesante todavía: augmentation *arregló* el *mode collapse* de la semilla 8 (70 % $\rightarrow$ 91.25 %) pero *introdujo* un nuevo outlier en la semilla 1 (95 % $\rightarrow$ 75 %). Es decir, *cambió qué semillas eran problemáticas* pero no eliminó la existencia de *basins* malos.

Interpretación: el problema no es falta de datos, sino la *inicialización aleatoria*. Augmentation reshapea ligeramente el paisaje de pérdida pero no resuelve la raíz. La solución de fondo es atacar la inicialización, lo que nos lleva al siguiente experimento.

5.4.2. Transfer learning con ResNet-18

Sustituyendo LeNet por una ResNet-18 [9] preentrenada en ImageNet [16] (con la última capa reemplazada para 16 clases y *fine-tuning* end-to-end con *learning rate* 10^{-4}), obtuvimos **97.62 % $\pm$ 1.63 %** en 10 corridas. Comparativa en la Tabla 3.

ResNet-18 supera al paper por **+1.44 pp** y reduce la varianza por un factor de **4.8 $\times$** . Ningún run cae por debajo de 93.75 % (versus 70 % del peor LeNet desde cero). La explicación coincide con la intuición de la Sección 2.8: los pesos preentrenados de ImageNet posicionan al modelo en un *basin* de buena calidad desde el inicio, eliminando la lotería de la inicialización aleatoria.

Cuadro 3: Resultados CNN sobre canonical 128×128 (10 corridas cada uno).

Modelo	Media	Std	Peor run	Mejor run
LeNet	91.25 %	7.83 %	70.00 %	97.50 %
LeNet + augmentation	91.62 %	6.71 %	75.00 %	100.00 %
ResNet-18 (transfer)	97.62 %	1.63 %	93.75 %	100.00 %
<i>Paper benchmark (100 runs)</i>	<i>96.18 %</i>	<i>n/r</i>	<i>n/r</i>	<i>n/r</i>

6. Pipeline YOLO

Habiendo establecido un baseline CNN sólido sobre imágenes preprocesadas, ahora atacamos el problema realista: clasificar directamente sobre la imagen completa, sin paso manual de recorte ni alineamiento.

6.1. Generación automática de anotaciones

YOLO requiere un dataset con **anotaciones de bounding boxes**: por cada imagen, un archivo de texto que indique para cada objeto la clase y las coordenadas ($x_c, y_c, ancho, alto$) normalizadas. LSA16 viene con *máscaras de segmentación* pero *no* con bounding boxes, así que tuvimos que generarlas automáticamente.

6.1.1. Iteración 1: detección por color (descartada)

El primer enfoque fue umbralización en espacio HSV: dado que los guantes son fluorescentes de color saturado, los píxeles “saturados” de la imagen raw son candidatos a guante. Usando el “color dominante” de la imagen segmentada como referencia per-imagen (porque el color de guante varía entre sujetos: algunos rosa/magenta, otros cian), buscamos esa banda de color en la imagen raw para localizar la mano.

Este enfoque **falló sistemáticamente en 76 de las 800 imágenes (9.5 %)**. La causa: las regiones de *piel facial saturada* (especialmente labios con lipstick rojizo) caen en el mismo rango de Hue del guante magenta, generando *bounding boxes* sobre *la cara* en lugar de la mano. El método de color es estructuralmente incapaz de discriminar mano y cara cuando comparten un Hue dominante similar.

6.1.2. Iteración 2: template matching enmascarado (adoptada)

Reemplazamos la heurística de color por **template matching** usando OpenCV. La idea es simple y robusta: dado que `lsa16_segmented_right_hand/` es literalmente un *recorte* de `lsa16_raw/` con el fondo seteado a negro, podemos buscar la posición exacta donde la imagen segmentada “encaja” dentro de la raw.

Concretamente, usamos `cv2.matchTemplate` con el método de *normalized cross-correlation* y una máscara binaria que indica qué píxeles del template son relevantes (los no-negros). El algoritmo desliza el template sobre la imagen raw y encuentra la posición de mejor coincidencia estructural –no solo de color– de los píxeles relevantes.

Resultados sobre las 800 imágenes:

- **794/800 (99.25 %) con bbox correctamente detectada.**
- 6 fallos: casos donde la imagen segmentada parece haber sido procesada levemente y no es un crop literal del raw.
- En las 76 imágenes donde el método de color era erróneo (cara), el template matching es siempre correcto. Verificación visual cruzada.

Lección metodológica: cuando existe *ground truth* indirecto disponible (en este caso, la segmented siendo un crop del raw), buscar coincidencia estructural es muy preferible a heurísticas basadas en color que pueden generar falsos positivos.

El dataset final (`dataset_yolo/`) tiene partición estratificada por clase 70/15/15: 556 train, 121 val, 117 test.

6.2. Configuración del entrenamiento

Usamos YOLOv8 (Ultralytics) con *transfer learning* desde pesos preentrenados en COCO. La Tabla 4 resume los hiperparámetros y el razonamiento detrás de cada uno.

Cuadro 4: Hiperparámetros del entrenamiento YOLOv8 y razonamiento.

Parámetro	Valor	Razón
Modelo	yolov8s.pt	balance velocidad/precisión, 11M parámetros (comparable a ResNet-18)
Pesos iniciales	COCO pretrained	transfer learning desde 1.28M imágenes de 80 clases
imgsz	640	default Ultralytics; mantiene suficiente detalle de los dedos
batch	32	limitado por la memoria de la GPU
epochs	100 (patience=20)	con early stopping de 20 épocas sin mejora
optimizer	AdamW (auto, lr=0.0005)	seleccionado automáticamente por Ultralytics
fliplr	0.0	CRÍTICO: flip horizontal cambia mano derecha por izquierda y rompe el ground truth
flipud	0.0	sin sentido geométrico para handshapes
mosaic	0.0	mosaic junta 4 imágenes en una; con una sola mano por imagen distorsionaría la relación espacial
degrees, translate, scale	10°, 0.1, 0.1	jitter geométrico leve
hsv_s, hsv_v	0.4, 0.3	variación moderada de iluminación

6.3. Hardware utilizado

Comparativa de costo computacional (entrenamiento de yolov8s, 88 épocas con early stop):

- Mac M3 Pro (CPU): aproximadamente 3 a 5 horas estimadas.
- NVIDIA RTX 3080 Ti (GPU): **8.5 minutos**.

Speedup de aproximadamente **20×** a favor de la GPU, consistente con la diferencia esperada entre CPU y GPU dedicada para redes neuronales. Para la tesis usamos los resultados de GPU. Esta diferencia ilustra que el entrenamiento es accesible en hardware modesto pero requiere paciencia; para iteración rápida una GPU es muy recomendable.

6.4. Resultados

La evaluación sobre el split de test (117 imágenes) calcula tanto métricas estándar de detección (mAP, IoU) como *classification accuracy* comparable con el baseline CNN (para cada imagen, tomamos la clase de la detección con mayor confianza y la comparamos con el ground truth). Resultados en la Tabla 5.

Interpretando: YOLOv8 detecta correctamente la mano en el 94.9% de las imágenes; en el 5.1% restante no produce detección (“miss”). Cuando detecta, clasifica correctamente al 94.59%. La calidad de localización es muy alta (IoU mediana 0.82, recordar que $\text{IoU} = 1$ es

Cuadro 5: Resultados de YOLOv8s sobre LSA16 raw (test set, 117 imágenes).

Métrica	Valor
Detection rate (% de imágenes donde detecta algo)	94.9 % (111/117)
Classification accuracy (sobre todas las test)	89.74 %
Classification accuracy (solo cuando detecta)	94.59 %
IoU promedio (calidad de localización)	0.779
IoU mediana	0.818
mAP@50	0.923
mAP@50-95	0.875
Precision	0.880
Recall	0.894

match perfecto). Las métricas estándar de detección (mAP@50 = 0.923) confirman que el modelo entrenado es sólido.

Las confusiones más frecuentes se dan entre clases visualmente similares: V → Horns (2 errores, ambas configuraciones con 2 dedos extendidos), Beak → Fingers/Index (1 error cada una, poses cerradas con punta visible). Estas confusiones reflejan ambigüedad intrínseca del dominio –un humano no entrenado en LSA también las confundiría– y no fallas del modelo.

6.5. Ablación: ¿más capacidad mejora el resultado?

Para testear si yolov8s era el modelo óptimo, probamos una variante más ambiciosa: yolov8m.pt (26M parámetros, aproximadamente el doble que yolov8s) con resolución de entrada de 960 píxeles por lado (más detalle). El entrenamiento tomó 39.7 minutos en GPU, casi cinco veces más que yolov8s. Resultados comparativos en la Tabla 6.

Cuadro 6: Ablación: yolov8s @ 640 vs yolov8m @ 960.

Métrica	yolov8s @ 640	yolov8m @ 960	Δ
Detection rate	94.9 %	98.3 %	+3.4 pp
Accuracy (todas las test)	89.74 %	90.60 %	+0.86 pp
Accuracy (solo detectadas)	94.59 %	92.17 %	−2.42 pp
IoU promedio	0.779	0.899	+12 pp
IoU mediana	0.818	0.949	+13 pp
mAP@50	0.923	0.925	≈
mAP@50-95	0.875	0.864	−1.1 pp
Tiempo entrenamiento	8.5 min	39.7 min	4.7×

El resultado tiene dos caras:

Lo que sí mejoró drásticamente. La calidad de localización (IoU mediana 0.82 → 0.95, las cajas están casi pegadas a la mano real) y la cobertura (de 6 a 2 *misses*). Más resolución y más capacidad ayudan a encontrar manos chicas o lejanas del frame.

Lo que no mejoró (incluso empeoró ligeramente). La accuracy de clasificación cuando el modelo detecta. Bajó 2.42 pp. Esto sugiere un **sobreajuste leve**: con 26 millones de parámetros y solo 556 imágenes de entrenamiento (aproximadamente 47 mil parámetros por sample de train), el modelo más grande comienza a memorizar características muy específicas de los ejemplos vistos, perdiendo capacidad de generalizar.

Conclusión metodológica: en datasets pequeños como LSA16, el cuello de botella ya no es la capacidad del modelo sino la *similitud visual intrínseca entre clases*. Más parámetros no resuelve eso. Para esta tesis reportamos **yolov8s como modelo principal** (mejor classification y más liviano) y yolov8m como ablación negativa –un resultado científicamente valioso que confirma que *más no siempre es mejor*.

7. Comparación experimental: CNN vs YOLO

La Tabla 7 consolida los resultados de todos los modelos experimentados sobre LSA16.

Cuadro 7: Comparación final de todos los modelos sobre LSA16.

Modelo	Input	Accuracy	Notas
LeNet (paper, 100 runs)	canonical 128×128	96.18 %	benchmark referencia
LeNet (nuestro, 10 runs)	canonical 128×128	91.25 % $\pm$ 7.83 %	varianza honesta
LeNet + aug (10 runs)	canonical 128×128	91.62 % $\pm$ 6.71 %	hipótesis refutada
ResNet-18 (10 runs)	canonical 128×128	97.62 % $\pm$ 1.63 %	supera al paper
LeNet sobre raw (10 runs)	raw 640×480	22.12 % $\pm$ 5.76 %	CNN colapsa
YOLOv8s	raw 640×480	89.74 %	modelo principal
YOLOv8m @ 960	raw 640×480	90.60 %	ablation: más $\neq$ mejor

7.1. Lectura comparativa

La tabla cuenta dos historias paralelas que vale la pena leer separadas.

Modelos sobre la mano preprocesada (canonical 128×128). En este eje, ResNet-18 (97.62 %) supera al baseline LeNet (91-96 %). Transfer learning resuelve la inestabilidad del CNN entrenado desde cero. La comparación se reduce a “*qué arquitectura clasifica mejor sobre un crop limpio*”. Escalar a una arquitectura más moderna y usar conocimiento previo (ImageNet) sí aporta una mejora cuantificable.

Modelos sobre el escenario realista (imagen completa 640×480 sin preprocesamiento). Aquí ocurre la diferencia cualitativa que da significado al trabajo:

- LeNet colapsa a **22.12 %**, apenas mejor que el azar ($1/16 = 6.25 %$) pero inviable como sistema real. Sin preprocesamiento manual, el CNN simplemente no puede operar.
- YOLOv8s alcanza **89.74 %**, una diferencia de **+67 puntos porcentuales** respecto a LeNet sobre la misma entrada.
- YOLOv8m apenas mejora a 90.60 %, sugiriendo que estamos cerca del techo realista del enfoque YOLO para este dataset.

La diferencia de +67 pp en el escenario realista es la **contribución principal del trabajo**. El cambio de paradigma de clasificación a detección espacial habilita el procesamiento de imágenes no controladas, donde el CNN tradicional fracasa.

7.2. Implicaciones para sistemas reales

Si el objetivo es desplegar un sistema de reconocimiento de LSA en una aplicación real (webcam de notebook, cámara de smartphone), las implicaciones del experimento son claras:

1. Un clasificador CNN tradicional **no es suficiente** a menos que se acompañe de un pipeline de visión clásica que recorte y alinee la mano antes de clasificar. Ese pipeline es frágil en condiciones reales (iluminación variable, fondo no blanco, distancia variable al sujeto).
2. Un detector espacial como YOLO opera **end-to-end sobre la imagen completa** sin necesidad de preprocesamiento manual, alcanzando una accuracy operativa de aproximadamente 90 % sobre nuestro test set.
3. YOLOv8s es lo suficientemente liviano (11M parámetros, ~22MB de pesos) para inferencia en tiempo real en hardware modesto.

8. Conclusiones y trabajo futuro

8.1. Conclusiones

Este trabajo presenta un estudio completo sobre LSA16 que va desde la replicación rigurosa del paper de referencia hasta una propuesta original basada en detección espacial. Los hallazgos principales son:

1. **Replicación reproducible del baseline:** identificamos el preprocesamiento canónico (alineamiento por PCA + corrección de inversión) como ingrediente crítico no documentado explícitamente en el paper de 2017. Sin él, LeNet alcanza solo 83 %; con él, 95.83 %.
2. **Caracterización honesta de la varianza:** 10 corridas revelan inestabilidad de entrenamiento que el promedio de 100 corridas del paper diluye. Es importante reportar la varianza, no solo la media.
3. **Mejora sobre el paper:** ResNet-18 con transfer learning alcanza $97.62\% \pm 1.63\%$, superando al paper en +1.44 pp con varianza $4.8\times$ menor. La intuición es que los pesos preentrenados de ImageNet posicionan al modelo en un *basin* bueno desde el inicio, eliminando la lotería de la inicialización aleatoria.
4. **Refutación de hipótesis sobre data augmentation:** aplicar augmentation no resuelve la inestabilidad de LeNet desde cero; cambia qué semillas son problemáticas pero no las elimina. El problema raíz es la inicialización aleatoria.
5. **Pipeline automático de anotaciones YOLO:** implementamos generación automática vía template matching enmascarado (99.25 % de éxito), evitando los falsos positivos sobre regiones faciales que produce la detección por color.
6. **YOLOv8s sobre raw alcanza 89.74 % de accuracy y mAP@50 = 0.923**, con **+67 pp sobre LeNet** en la misma entrada. *Esta es la contribución central del trabajo:* detección espacial supera a clasificación tradicional cuando operamos sobre imágenes no preprocesadas.
7. **Ablación negativa con yolov8m+imgsz=960:** mejora drásticamente la localización (IoU mediana $0.82 \rightarrow 0.95$) pero *no* la clasificación (sobreajuste leve). Confirma que en datasets pequeños, más capacidad no siempre ayuda.

La hipótesis central del trabajo se valida: **en escenarios realistas no preprocesados, la detección espacial supera ampliamente a la clasificación de imagen entera**. Esto justifica la elección de YOLO (o detectores equivalentes) como base para sistemas reales de reconocimiento de LSA.

8.2. Trabajo futuro

- **Evaluación LOSO** (*leave-one-subject-out*) sobre todos los modelos: en cada fold, se deja a un sujeto entero fuera del entrenamiento y se evalúa sobre él. Es un protocolo más representativo del uso real con usuarios nuevos (típicamente baja la accuracy 10-15 pp pero es más honesto).
- **ResNet-18 + data augmentation**: combinación no testeada que podría llevar la accuracy CNN cerca del 99 %, a un costo computacional adicional de aproximadamente 90 minutos en CPU.
- **Variante YOLOv8n (nano)**: modelo aún más liviano (3M parámetros) para deployment en tiempo real en hardware modesto como un notebook o un smartphone.
- **Test end-to-end con webcam real**: validar el pipeline en condiciones no controladas (iluminación variable, fondo no blanco, distancia variable al sujeto).
- **Escalamiento a video y handshapes dinámicos**: el dataset LSA64 [3] contiene secuencias temporales de señas completas. Requiere arquitecturas espacio-temporales (CNNs 3D o LSTM sobre features YOLO) que son la extensión natural de este trabajo.

Referencias

- [1] F. Quiroga, R. Antonio, F. Ronchetti, L. Lanzarini, A. Rosete, *A Study of Convolutional Architectures for Handshape Recognition applied to Sign Language*, XXIII Congreso Argentino de Ciencias de la Computación, 2017.
- [2] F. Ronchetti, F. Quiroga, L. Lanzarini, C. Estrebou, *Handshape Recognition for Argentinian Sign Language using ProbSom*, Journal of Computer Science and Technology 16(1):1–5, 2016.
- [3] F. Ronchetti, F. Quiroga, C. Estrebou, L. Lanzarini, A. Rosete, *LSA64: An Argentinian Sign Language Dataset*, XXII Congreso Argentino de Ciencias de la Computación, 2016.
- [4] F. Ronchetti, F. Quiroga, C. Estrebou, L. Lanzarini, A. Rosete, *Sign Language Recognition Without Frame-Sequencing Constraints: A Proof of Concept on the Argentinian Sign Language*, Springer LNCS, pp. 338–349, 2016.
- [5] H. Cooper, B. Holt, R. Bowden, *Sign Language Recognition*, Visual Analysis of Humans, Springer London, pp. 539–562, 2011.
- [6] Y. LeCun, L. Bottou, Y. Bengio, P. Haffner, *Gradient-based learning applied to document recognition*, Proceedings of the IEEE 86(11):2278–2324, 1998.
- [7] K. Simonyan, A. Zisserman, *Very Deep Convolutional Networks for Large-Scale Image Recognition*, arXiv:1409.1556, 2014.
- [8] J.T. Springenberg, A. Dosovitskiy, T. Brox, M. Riedmiller, *Striving for Simplicity: The All Convolutional Net*, arXiv:1412.6806, 2014.
- [9] K. He, X. Zhang, S. Ren, J. Sun, *Deep Residual Learning for Image Recognition*, IEEE CVPR, pp. 770–778, 2016.
- [10] C. Szegedy, V. Vanhoucke, S. Ioffe, J. Shlens, Z. Wojna, *Rethinking the Inception Architecture for Computer Vision*, IEEE CVPR, 2016.
- [11] J. Redmon, S. Divvala, R. Girshick, A. Farhadi, *You Only Look Once: Unified, Real-Time Object Detection*, IEEE CVPR, 2016.

- [12] J. Redmon, A. Farhadi, *YOLO9000: Better, Faster, Stronger*, IEEE CVPR, 2017.
- [13] J. Redmon, A. Farhadi, *YOLOv3: An Incremental Improvement*, arXiv:1804.02767, 2018.
- [14] G. Jocher, A. Chaurasia, J. Qiu, *Ultralytics YOLOv8*, <https://github.com/ultralytics/ultralytics>, 2023.
- [15] T.-Y. Lin, M. Maire, S. Belongie, J. Hays, P. Perona, D. Ramanan, P. Dollár, C.L. Zitnick, *Microsoft COCO: Common Objects in Context*, ECCV, 2014.
- [16] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, L. Fei-Fei, *ImageNet: A Large-Scale Hierarchical Image Database*, IEEE CVPR, 2009.
- [17] D.P. Kingma, J. Ba, *Adam: A Method for Stochastic Optimization*, arXiv:1412.6980, 2014.
- [18] S. Ioffe, C. Szegedy, *Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift*, ICML, 2015.
- [19] D.-A. Clevert, T. Unterthiner, S. Hochreiter, *Fast and Accurate Deep Network Learning by Exponential Linear Units (ELUs)*, arXiv:1511.07289, 2015.